

Sliding Window Pattern in DSA – Complete Theory (Exam & Interview)

1. What is Sliding Window Pattern?

Sliding Window is an optimization technique used to process a range (window) of elements in arrays or strings. Instead of recalculating values for every subarray, the window slides forward and updates the result efficiently.

2. When to Use Sliding Window

Use this pattern when:

- You are asked about subarrays or substrings
- The problem involves a contiguous sequence
- Fixed-size or variable-size window is involved
- You need optimal $O(n)$ solutions

3. Types of Sliding Window

A. Fixed Size Window:

- Window size k is given
- Example: Maximum sum subarray of size k

B. Variable Size Window:

- Window size changes dynamically
- Example: Longest substring with k distinct characters

4. Core Sliding Window Workflow

1. Initialize window start and end
2. Expand window by moving end pointer
3. Update window condition
4. Shrink window if condition breaks
5. Store best answer

5. Sliding Window Flowchart (Text Diagram)

START → Initialize start=0 → Expand end pointer → Update condition → Condition valid? → YES: update answer → NO: shrink window → Repeat → END

6. Fixed Window Pseudocode

```
window_sum = 0
```

```
for end in range(n):
```

```
    window_sum += arr[end]
```

```
    if end >= k-1:
```

```
        answer = max(answer, window_sum)
```

```
        window_sum -= arr[start]
```

```
        start += 1
```

7. Variable Window Pseudocode

```
start = 0
```

```
for end in range(n):
```

```
    add arr[end]
```

```
    while condition breaks:
```

```
        remove arr[start]
```

```
        start += 1
```

```
    update answer
```

8. Python Example: Maximum Sum Subarray of Size K

```
def maxSumSubarray(nums, k):
```

```
    window_sum = 0
```

```
    max_sum = 0
```

```
    start = 0
```

```
    for end in range(len(nums)):
```

```
        window_sum += nums[end]
```

```
        if end >= k - 1:
```

```
            max_sum = max(max_sum, window_sum)
```

```
    window_sum -= nums[start]

    start += 1

return max_sum
```

9. Dry Run Example

nums = [2,1,5,1,3,2], k = 3

Window [2,1,5] sum=8

Slide → [1,5,1] sum=7

Slide → [5,1,3] sum=9 (max)

Slide → [1,3,2] sum=6

10. Common Sliding Window Problems

1. Maximum / Minimum Sum Subarray of Size K
2. Longest Substring Without Repeating Characters
3. Longest Substring with K Distinct Characters
4. Fruits into Baskets
5. Minimum Window Substring
6. Permutation in String
7. Average of Subarrays

11. Time & Space Complexity

Time Complexity: $O(n)$

Space Complexity: $O(1)$ or $O(k)$ depending on data structure

12. Common Interview Questions & Answers

Q1. Why Sliding Window is better than brute force?

Ans: Brute force recalculates subarrays repeatedly ($O(n^2)$), sliding window avoids repetition ($O(n)$).

Q2. Difference between Fixed and Variable Sliding Window?

Ans: Fixed window has constant size, variable window adjusts size dynamically.

Q3. Sliding Window vs Two Pointer?

Ans: Sliding Window works on contiguous subarrays, two pointer can work on non-contiguous pairs.

Q4. When does Sliding Window fail?

Ans: When subarrays are not contiguous or problem requires random access.

13. Interview Explanation Strategy

Explain brute force → show redundancy → introduce sliding window → explain window expansion & shrink → complexity analysis